机器学习 (六)

机器学习 (六) 决策树 实验报告

学院:	智能工程学院	专业:	智能科学与技术
姓名:	林楗棋	学号:	24363050

目录

1 第一部分：函数介绍 **2**

2 编写函数 **2**

3 构建决策树 **7**

函数介绍

本部分介绍了两个函数的实现原理与作用，分别为 **Counter** 与 **np.unique**。Counter 对第二列频次进行统计，返回键为取值、值为次数的字典结构。访问不存在的键会返回默认值 0，因此可以直接判断某类别是否出现。通过 **values** 和列表化的键，可以快速获得频次分布和唯一取值，为后续统计和分裂计算打基础。**np.unique** 可以对整体数组或指定列去重，并返回排序后的唯一值。

编写函数

题目 2.1： 该数据集 (train_titanic.csv) 为分类数据集，为泰坦尼克号的部分乘客信息以及最后是否生还。包括了四个属性值以及一个标记属性 (即为 Survived 类型，代表是否生还)，属性信息分别为 Sex(性别)，sibsp(堂兄弟妹个数)，Parch(父母与小孩的个数)，Pclass(乘客等级)。其中该数据集无缺失值和异常值，且所有连续变量已自动转换为离散变量，标记 (Survived 类型) 也自动转变为离散变量 0 和 1，所以你无需进行数据预处理，可以直接使用该数据集。

题目 2.2： 使用 pandas 库将训练数据集'train_titanic.csv' 载入到 Dataframe 对象中

Listing 1: 读取训练集并拆分特征与标签

```python
import pandas as pd
import numpy as np

dataTrain = pd.read_csv("train_titanic.csv")
dfTrain = pd.DataFrame(dataTrain)
X = dfTrain.iloc[:, :-1].to_numpy()
y = dfTrain.iloc[:, -1:].to_numpy()
```

和前几次实验一样，使用 **pd.read_csv** 读取数据后转换为 DataFrame。将特征与标签按列拆分，使后续函数输入格式统一。

题目 2.3： 编写函数，给定任何标记数组计算其信息熵输入：标记数组输出：该数组对应的信息熵计算信息熵公式：某数组包含 K 个不同的取值，样本为第 $k(k=1,2,\ldots,K)$ 个值的数量所占比例为 p_k，则其信息熵为

$$Ent = -\sum_{k=1}^{K} p_k \log_2 p_k$$

例：输入：`[[0],[1]]` 输出：$(-\frac{1}{2}\log_2\frac{1}{2}) + (-\frac{1}{2}\log_2\frac{1}{2}) = 1$

Listing 2: 信息熵函数

```python
from collections import Counter
import numpy as np

```

```python
def entropy(label):
    labels = np.asarray(label).reshape(-1)
    total = labels.shape[0]
    counter = Counter(labels)
    probs = np.array(list(counter.values())) / total
    ent = -np.sum(probs * np.log2(probs))
    return ent
```

由于计算信息熵只需要获取标签的频次分布，因此使用 Counter 得到每一个值的频次，除以总样本数得到概率。该实现先将标签展平成一维，再统计各类别频次。用频次除以总样本数得到概率，按信息熵公式求和即可。使用 **np.log2** 保证对数底为 2，并能自然处理多类别情况。

题目 2.4： 编写函数，函数功能为将所给的数据集按照指定维度 dimension 进行划分为若干个不同的数据集【输入】：属性集合，标记集合，维度索引【输出】：划分后所得到的子树属性集合，子树标记集合

例子：【输入】：feature(属性值集合)：`[[0,0,0],[0,0,1],[1,0,2]]` label(标记集合)：`[[0],[1],[2]]` 划分维度：0 【输出】：`[[0,0,0],[0,0,1]]` 和 `[[1,0,2]]` `[[0],[1]]` 和 `[[2]]` tips：即将 feature 按其第 0 维度进行划分，由于 feature 的 0 维有 0 和 1 两个不同的数值，所以 feature 划分为 `[[0,0,0],[0,0,1]]` 和 `[[1,0,2]]`，label 划分为 `[[0],[1]]` 和 `[[2]]`

```python
def split(feature, label, dimension):
    fea1 = np.unique(feature[:, dimension])
    split_feature = []
    split_label = []
    for fea in fea1:
        idx = feature[:, dimension] == fea
        split_feature.append(feature[idx])
        split_label.append(label[idx])
    return split_feature, split_label
```

函数先获取指定维度的唯一取值，再用布尔索引划分不同的子特征集和子标签集。每个取值生成一个子特征集与子标签集，便于后续计算条件熵。返回列表结构使遍历与汇总更方便。

题目 2.5： 编写函数，函数功能为进行【一次】决策树的结点划分，遍历找出该属性集合中信息增益(使用 ID3 算法中的公式计算)【最大】的属性输入：属性集合，标记集合输出：该次划分的最佳信息增益，最佳划分维度计算信息增益公式：某数据集 D 有若干属性值以及对应的标记值，其总样本大小为 $|D|$。这里取其中一个属性类型 feature，该特征包含 V 个不同的取值，属性值为第 $v(v=1,2,\ldots,V)$ 个值的数量为 $|D^v|$，$\sum_{v=1}^{V}|D^v|=|D|$，则该属性值对应的信息增益为

$$Gain(D,\text{feature}) = Ent(D) - \sum_{v=1}^{V}\frac{|D^v|}{|D|}Ent(D^v)$$

所以该函数中你需要计算出每个属性值的信息增益并输出，然后返回最大的信息增益并返回该特征的维数以及最大的信息增益值。

Listing 3: 一次 ID3 划分

```python
def one_split_ID3(feature, label):
    entro = entropy(label)
    gain = []
    for i in range(feature.shape[1]):
        fea, lab = split(feature, label, i)
        ent = 0
        for I, k in enumerate(lab):
            ent += fea[I].shape[0] / feature.shape[0] * entropy(k)
        ent = entro - ent
        gain.append(ent)
    best_entropy = max(gain)
    best_dimension = gain.index(best_entropy)
    return best_entropy, best_dimension
```

ID3 先计算总体熵，再利用前面编写的划分函数遍历各特征的条件熵并取差值作为信息增益。信息增益通过计算对不同子集的熵加权计算，信息增益越大，说明该特征对样本不确定性降低越多。函数返回最大增益及其维度，供树结点划分使用。

题目 2.6： 编写函数，函数功能为进行【一次】决策树的结点划分，遍历找出该属性集合中信息增益率 (使用 C4.5 算法中的公式计算)【最大】的属性输入：属性集合，标记集合输出：该次划分的信息增益率，最佳维度计算信息增益率公式：某数据集 D 有若干属性值以及对应的标记值，其总样本大小为 $|D|$。这里取其中一个属性类型 feature，该属性包含 V 个不同的取值，属性值为第 $v(v=1,2,\ldots,V)$ 个值的数量为 $|D^v|$，$\sum_{v=1}^{V}|D^v|=|D|$。该属性本身的信息熵为 $Ent(\text{feature})$，则该属性值对应的信息增益率为

$$Gain_{\text{ratio}}(D,\text{feature})=\frac{Gain(D,\text{feature})}{Ent(\text{feature})}$$

所以该函数中你需要输出每个特征的信息增益率，之后返回最大的信息增益率对应的属性维数以及最大的信息增益率。

Listing 4: 一次 C4.5 划分

```python
def one_split_C4_5(feature, label):
    base = entropy(label)
    best_ratio = float('-inf')
    best_dimension = None
    total = feature.shape[0]

    for dim in range(feature.shape[1]):
        fea, lab = split(feature=feature, label=label, dimension=dim)
        cond_ent = 0.0
        i = 0.0
        for sf, sl in zip(fea, lab):
            p = sf.shape[0] / total
```

```
            cond_ent += p * entropy(sl)
            if p > 0:
                i -= p * np.log2(p)
        gain = base - cond_ent
        ratio = gain / i if i > 0 else 0
        if ratio > best_ratio:
            best_ratio = ratio
            best_dimension = dim

    return best_ratio, best_dimension
```

C4.5 在信息增益上除以特征集子集的信息熵。因为，我们希望子集中尽量只有一个种类的特征值，当子集的特征值越少，其熵就越小，分数就越大，因此可以减少取值数多，划分细的特征带来的偏好。比如说按 ID 分类就会计算出熵增益很大，但是这种分类对类别纯度提升不一定有帮助，所以引入信息增益率来修正。代码同时计算条件熵与 IV，并在 IV 为 0 时做保护。最终返回信息增益率最高的维度，作为最优划分特征。

题目 2.7: 编写函数，进行【一次】决策树的结点划分，遍历找出该属性集合中基尼系数 (使用 CART 算法中的公式计算) 最小的属性以及最佳的划分值输入：属性集合，标记集合输出：该次划分的最佳的基尼系数，最佳维度，最佳划分值计算基尼系数公式：某数据集 D 有若干属性值以及对应的标记值，其总样本大小为 $|D|$，该集合中样本标记类别总共有 K 类，第 k 类样本所占比例为 $p_k(k=1,2,\ldots,K)$，则该数据集对应的基尼系数为

$$Gini(D) = 1 - \sum_{k=1}^{K} p_k^2$$

而取其中一个属性类型 feature，选定该类型的一个值 value，根据 feature 这个属性是否为 value 将数据集分为两个子集 D_1 和 D_2，则该属性 feature 对应的基尼系数为

$$Gini_{\text{index}}(D, \text{feature}) = \frac{|D_1|}{|D|} Gini(D_1) + \frac{|D_2|}{|D|} Gini(D_2)$$

Listing 5: 一次 CART 划分

```
def one_split_CART(feature, label):
    def gini(sub_label):
        labels = np.asarray(sub_label).reshape(-1)
        total = labels.shape[0]
        counter = Counter(labels)
        return 1 - sum((cnt / total) ** 2 for cnt in counter.values())

    best_gini = float('inf')
    best_dimension = None
    best_value = None
    total = feature.shape[0]

    for dim in range(feature.shape[1]):
```

```
        for val in np.unique(feature[:, dim]):
            left_idx = feature[:, dim] <= val
            right_idx = ~left_idx
            if left_idx.sum() == 0 or right_idx.sum() == 0:
                continue
            g_left = gini(label[left_idx])
            g_right = gini(label[right_idx])
            g_split = (left_idx.sum() / total) * g_left + (right_idx.sum() / total) * g_right

            if g_split < best_gini:
                best_gini = g_split
                best_dimension = dim
                best_value = val

    return best_gini, best_dimension, best_value
```

CART 使用二分划分并以基尼指数为准则，遍历每个特征的所有候选取值。基尼指数在越纯的时候越小，对每个候选划分计算左右子集的加权基尼，从而衡量划分纯度。最终选择基尼最小的维度与阈值作为最优划分。

题目 2.8： 应用之前你在第 4、5、6 个部分编写的三个函数，在训练数据集'train_titanic.csv' 上依次使用这些函数进行【一次】结点划分，并输出对应的最佳属性维数以及相应的信息增益值/信息增益率/(基尼系数和分类值)

Listing 6: 三种方法的一次划分结果

```
import pandas as pd
import numpy as np

file = pd.read_csv("train_titanic.csv")
df = pd.DataFrame(file)
feature = np.array(df)[:, :-1]
label = np.array(df)[:, -1]
e1, d1 = one_split_ID3(feature, label)
e2, d2 = one_split_C4_5(feature, label)
e3, d3, v3 = one_split_CART(feature, label)
print(e1, d1, e2, d2, e3, d3, v3)
```

```
0.10750711887455178 0 0.11339165967945304 0 0.2964915724641573 0 0
```

三种方法在一次划分中都选择了第 0 列特征 Sex 作为最优属性。信息增益与信息增益率数值表明性别对标签不确定性降低明显。CART 的最优划分值也落在该维度，说明该特征对类别纯度提升最大。

构建决策树

题目 3.1： 用【ID3】算法实现一棵完整的决策树。完成 DTree 类中的 TreeGenerate、train 函数以完成决策树的构建。并完成 DTree 类中的 predict 函数来用构建好的决策树来对测试数据集进行预测并输出预测准确率。

Listing 7: 训练并评估模型

```python
import pandas as pd

train_frame = pd.read_csv("train_titanic.csv")
dt = DTree()
dt.train(train_frame.to_numpy())

test_frame = pd.read_csv("test_titanic.csv")
acc = dt.predict(test_frame.to_numpy())
print(acc)
```

```
0.8415841584158416
```

按照注释和课件填充函数，调用先前写的函数。TreeGenerate 按 ID3 准则递归选择最优特征，遇到纯标签或无可用特征时生成叶结点。每个结点保存多数类标签，用于分支缺失时的默认预测。训练阶段预先记录每个属性可能取值，使树分支完整且可遍历。预测阶段沿分支搜索直至叶结点，最终准确率约为 84.16%。